

Datapath Design Using LPM Components

Emmanuel Cano

Goal of the Exercise

The goal of this exercise is to design a simplified CPU-style datapath in VHDL using LPM components available in Quartus. The design performs arithmetic operations (addition and subtraction) on operands stored in SRAM, temporarily stores the results in registers, and writes the final value back to SRAM. We then use ModelSim to verify correct data movement between the components and correct arithmetic operations as well.

Tools and Environment

- HDL: VHDL
- FPGA Toolchain: Intel Quartus Prime Lite
- Simulator: ModelSim
- Target Platform: Simulation-only (no hardware deployment)

VHDL Implementation

```
library ieee;
use ieee.std_logic_1164.all;

entity D_FlipFlop_32bit is
    Port (
        clk    : in  std_logic;
        load    : in  std_logic;
        d       : in  std_logic_vector(31 downto 0);
        q       : out std_logic_vector(31 downto 0)
    );
end entity;

architecture D_FlipFlop_32bitArch of D_FlipFlop_32bit is
    signal temp : std_logic_vector(31 downto 0);
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if load = '1' then
                temp <= d;
            end if;
        end if;
    end process;
    q <= temp;
end architecture;
```

Figure 1. *D_FlipFlop_32bit* VHDL implementation

Figure 1's D_FLIP_FLOP holds 32-bits of data and is able to output all of those 32-bits as the variable q. It is load enabled which ensures that on a rising edge it loads the data as long as the load is '1'. All data transfers will occur on rising clock edges. Unlike in previous exercises, this and the following designs will include clocked sequential logic in the form of registers and memory.

ADD_SUB Unit (LPM_ADD_SUB)

Generated from IP Catalog to support 32-bit unsigned addition and subtraction.

Settings:

- Width = 32
- Representation = Unsigned
- Add/Sub Control = Yes
- Overflow Output= Yes

The add_sub control input selects the operation:

- 1 → Addition
- 0 → Subtraction

The unit also provides an overflow flag.

SRAM Unit (RAM: 1-Port)

A 256×32-bit single-port RAM instance generated from the IP Catalog.

Settings:

- Width = 32
- Depth = 256
- Initialized with memory.mif file

```
DEPTH = 256;
WIDTH = 32;

ADDRESS_RADIX = HEX;
DATA_RADIX = HEX;

CONTENT
BEGIN
    00 : FFFFFFFF; -- first operand
    01 : 00000001; -- second operand
END;
```

Figure 2. *memory.mif*

Register Unit (LPM_SHIFTREG)

This was generated using the IP Catalog.

Settings:

- Width = 32
- Use Parallel Load = yes
- Clock Enable = yes

Since the registers are used only for parallel loading, the clock enable port allows for the shifting to be disabled.

```
library ieee;
use ieee.std_logic_1164.all;

entity toplevel is
  port(
    clk      : in  std_logic;
    loadA    : in  std_logic;
    loadB    : in  std_logic;
    loadRes  : in  std_logic;
    add_sub  : in  std_logic;
    addrA    : in  std_logic_vector(7 downto 0);
    addrB    : in  std_logic_vector(7 downto 0);
    addrRes  : in  std_logic_vector(7 downto 0);
    overflow : out std_logic;

    -- debug outputs
    regA_out_dbg : out std_logic_vector(31 downto 0);
    regB_out_dbg : out std_logic_vector(31 downto 0);
    result_dbg   : out std_logic_vector(31 downto 0)
  );
end entity;

architecture toplevelarch of toplevel is

  -- SRAM outputs
  signal sram_outA : std_logic_vector(31 downto 0);
  signal sram_outB : std_logic_vector(31 downto 0);

  -- Register outputs
  signal regA_out  : std_logic_vector(31 downto 0);
  signal regB_out  : std_logic_vector(31 downto 0);

  -- ADD/SUB output
  signal alu_result : std_logic_vector(31 downto 0);

  -- Output of the DFF result register
  signal result_reg : std_logic_vector(31 downto 0);

begin
  -----
```

```

-- READ Operands from SRAM
-----
RAM_A : entity work.sram_lport
  port map (
    address => addrA,
    clock   => clk,
    data    => (others=>'0'),
    wren    => '0',
    q       => sram_outA
  );

RAM_B : entity work.sram_lport
  port map (
    address => addrB,
    clock   => clk,
    data    => (others=>'0'),
    wren    => '0',
    q       => sram_outB
  );

-----
-- OPERAND REGISTERS
-----
RA : entity work.shiftreg
  port map (
    clock => clk,
    enable => loadA,
    load  => '1',
    data  => sram_outA,
    q     => regA_out
  );

RB : entity work.shiftreg
  port map (
    clock => clk,
    enable => loadB,
    load  => '1',
    data  => sram_outB,
    q     => regB_out
  );

-----
-- ADDER / SUBTRACTOR UNIT
-----
U_ADD : entity work.add_sub_32bit
  port map (
    dataa  => regA_out,
    datab => regB_out,
    add_sub => add_sub,
    result => alu_result,
    overflow => overflow
  );

-----
-- RESULT REGISTER
-----
U_REG : entity work.D_FlipFlop_32bit
  port map (
    clk  => clk,
    load => loadRes,
    d    => alu_result,
    q    => result_reg
  );

```

```

);

-----
-- WRITE BACK TO SRAM
-----

RAM_RES : entity work.sram_lport
  port map (
    address => addrRes,
    clock   => clk,
    data    => result_reg,
    wren    => loadRes,
    q       => open
  );

-----
-- DEBUG OUTPUTS
-----

regA_out_dbg <= regA_out;
regB_out_dbg <= regB_out;
result_dbg   <= result_reg;

end architecture;

```

Figure 3. *toplevel VHDL implementation*

Figure 3 is the top level VHDL code that allows the D Flip-Flop and the LPM components to work together. The system datapath is: SRAM → Register A/B (LPM_SHIFTREG) → ADD/SUB → Result Register (D_FLIP_FLOP) → SRAM. When LoadA or LoadB is asserted, operand values from SRAM are loaded into operand registers. When LoadRes is asserted, the result from the adder/subtractor is written into the D_FLIP_FLOP and then stored back into SRAM.

Testbench Design

```

library ieee;
use ieee.std_logic_1164.all;

entity testbench is
end entity testbench;

architecture testbenchArch of testbench is
  signal clk      : std_logic := '0';
  signal loadA    : std_logic := '0';
  signal loadB    : std_logic := '0';
  signal loadRes  : std_logic := '0';
  signal add_sub  : std_logic := '1';
  signal addrA    : std_logic_vector(7 downto 0) := x"00";
  signal addrB    : std_logic_vector(7 downto 0) := x"01";
  signal addrRes  : std_logic_vector(7 downto 0) := x"02";
  signal overflow : std_logic;

  signal regA_dbg : std_logic_vector(31 downto 0);
  signal regB_dbg : std_logic_vector(31 downto 0);
  signal result_dbg : std_logic_vector(31 downto 0);

```

```

begin
    clk <= not clk after 10 ns;

    DUT : entity work.toplevel
        port map(
            clk          => clk,
            loadA         => loadA,
            loadB         => loadB,
            loadRes       => loadRes,
            add_sub       => add_sub,
            addrA         => addrA,
            addrB         => addrB,
            addrRes       => addrRes,
            overflow      => overflow,
            regA_out_dbg  => regA_dbg,
            regB_out_dbg  => regB_dbg,
            result_dbg    => result_dbg
        );

process
begin
    wait for 20 ns;

    -----
    -- TEST 1: Overflow Test + ADD Operation
    -----

    addrA <= x"00";
    addrB <= x"01";
    wait for 40 ns;

    loadA <= '1';
    loadB <= '1';
    wait for 20 ns;
    loadA <= '0';
    loadB <= '0';

    add_sub <= '1';
    wait for 20 ns;

    addrRes <= x"02";
    loadRes <= '1';
    wait for 20 ns;
    loadRes <= '0';

    wait for 40 ns;

    -----
    -- TEST 2: SUB operation
    -----

    add_sub <= '0';
    wait for 20 ns;

    addrRes <= x"03";
    loadRes <= '1';
    wait for 20 ns;
    loadRes <= '0';

    wait for 40 ns;

    -----
    -- TEST 3: Same Address Test

```

```

-----

addrA <= x"01";
addrB <= x"01";
wait for 40 ns;

loadA <= '1';
loadB <= '1';
wait for 20 ns;
loadA <= '0';
loadB <= '0';

add_sub <= '1';
wait for 20 ns;

addrRes <= x"04";
loadRes <= '1';
wait for 20 ns;
loadRes <= '0';

wait for 100 ns;

wait;
end process;
end testbenchArch;

```

Figure 4. Adder/Subtractor testbench design

The testbench applies the control signals needed to:

- Load two operands from memory
- Perform addition and store the result
- Perform subtraction and store the result
- Load large values and demonstrate overflow
- Load from the same address

Block Diagrams

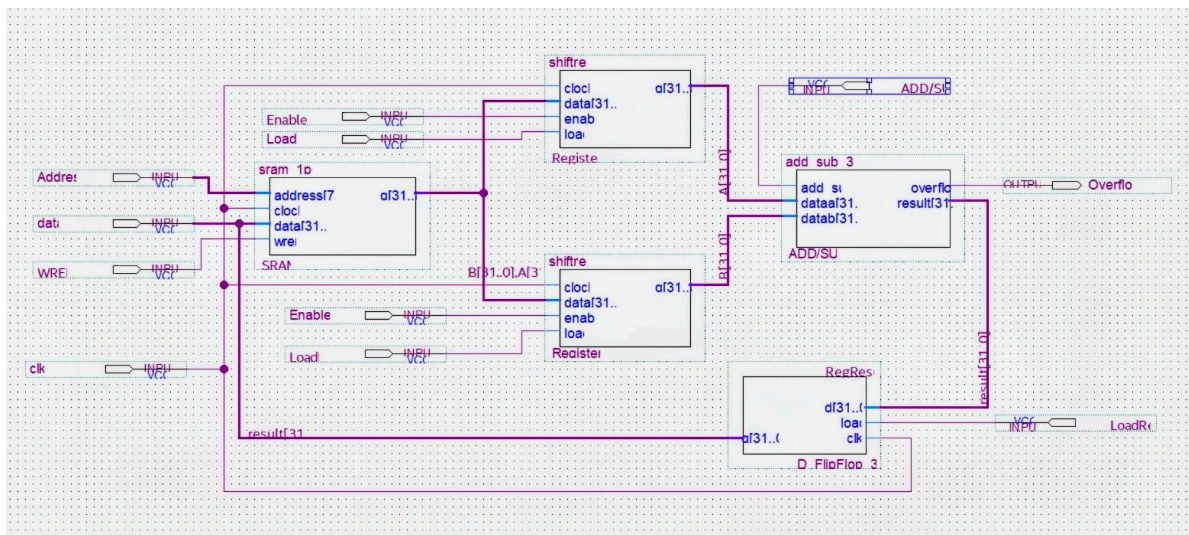


Figure 5. Adder/Subtractor Block Diagram

Figure 5 follows the same system datapath as the top level VHDL code: SRAM → Register A/B (LPM_SHIFTREG) → ADD/SUB → Result Register (D_FLIP_FLOP) → SRAM.

ModelSim Simulation Results

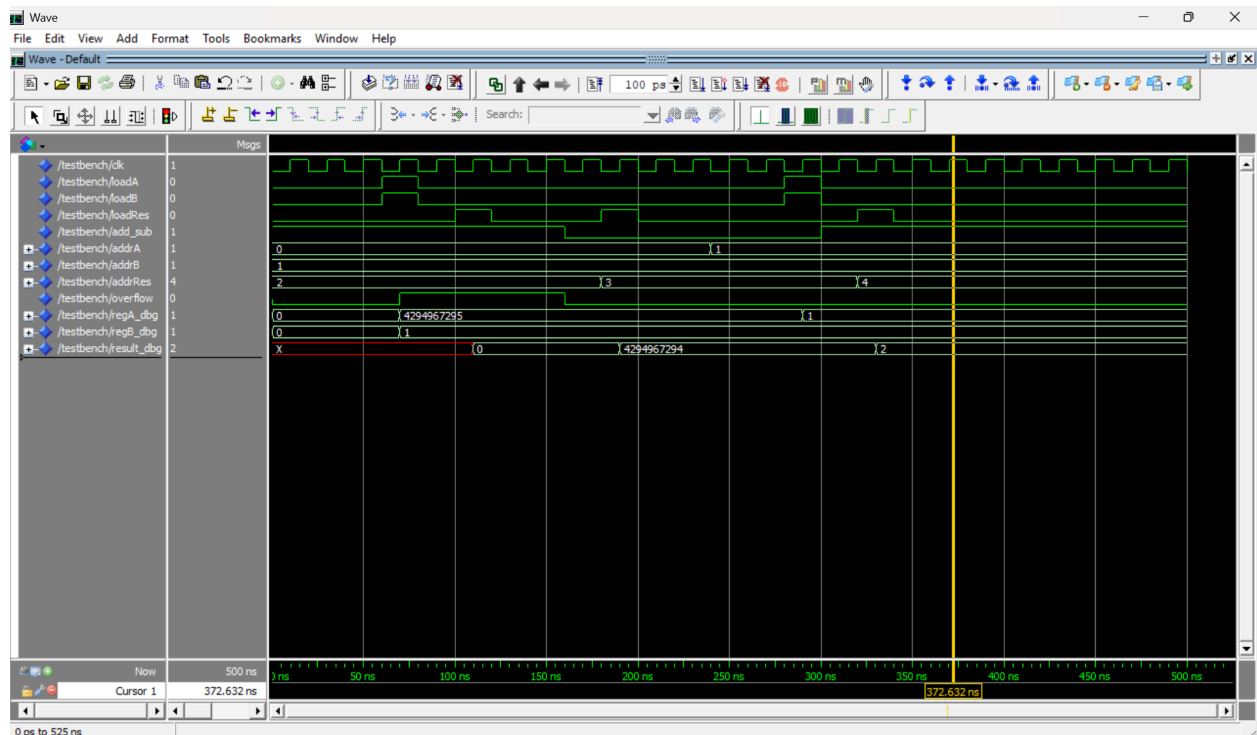


Figure 6. Adder/Subtractor Wave

Overflow + Addition Test

- Operand A loaded from address 0 with value 0xFFFFFFFF on a rising edge.
- Operand B loaded from address 1 with value 0x00000001 on a rising edge.
- `add_sub = 1` triggers addition which produces: $0xFFFFFFFF + 0x00000001 = 0x00000000$ with overflow = 1
- Result written to address 2 on a rising edge when `LoadRes = 1`.
- The first test correctly added and detected overflow.

Subtraction Test

- With `add_sub = 0`, subtraction was performed and stored into memory at location 3 on a rising edge when `LoadRes = 1`.
- Correctly subtracted the values of $0xFFFFFFFF - 0x00000001 = 0xFFFFFFFF$

Same Address Test

- Both operands A and B loaded from address 1 with value + 0x00000001 on a rising edge.
- add_sub = 1 triggers addition which produces: $0x00000001 + 0x00000001 = 0x00000002$
- Result written to address 4 on a rising edge when LoadRes = 1.
- Correctly loaded same address values into both operands A and B followed by correct addition.

Conclusion

In this exercise, a simplified CPU-style datapath was successfully implemented using VHDL and Quartus LPM components. The system demonstrated the ability to load operands from memory, store intermediate values in registers, perform both addition and subtraction, and write the final results back to SRAM. ModelSim simulation verified correct datapath as well as correct arithmetic overflow detection. The use of LPM components greatly simplified the design process and modeled how real microprocessors use reusable modules such as ALUs, registers, and memory blocks.